

## TP – J2EE / Jakarta EE

Année universitaire 2025–2026

---

# Gestion des Produits

## Spring MVC + Hibernate

### Objectif

Réaliser une application web dynamique permettant la gestion des produits en utilisant Spring MVC, Hibernate, JSP/JSTL, MySQL et Apache Tomcat.

Réalisé par :  
Chaima Menouar

Encadré par :  
Pr. Mohamed CHERRADI

---

# Introduction

Dans ce TP, nous avons développé une application web dynamique pour la gestion des produits. L'application permet d'effectuer les opérations CRUD suivantes :

- Ajouter un produit ;
- Afficher la liste des produits ;
- Modifier un produit ;
- Supprimer un produit.

Le projet est basé sur l'architecture MVC avec Spring MVC, et utilise Hibernate pour la communication avec la base de données MySQL.

**Technologies utilisées :** Java, Maven, Spring MVC, Hibernate, JSP/JSTL, MySQL, Apache Tomcat 9.

# Table des matières

|                                            |           |
|--------------------------------------------|-----------|
| <b>Introduction</b>                        | <b>1</b>  |
| <b>1 Présentation du projet</b>            | <b>3</b>  |
| <b>2 Architecture de l'application</b>     | <b>4</b>  |
| 2.1 Conception générale . . . . .          | 4         |
| <b>3 Structure du projet</b>               | <b>5</b>  |
| <b>4 Configuration du projet</b>           | <b>6</b>  |
| 4.1 Fichier pom.xml . . . . .              | 6         |
| 4.2 Fichier web.xml . . . . .              | 6         |
| 4.3 Configuration Hibernate . . . . .      | 7         |
| <b>5 Description des composants</b>        | <b>8</b>  |
| 5.1 Model . . . . .                        | 8         |
| 5.2 DAO . . . . .                          | 8         |
| 5.3 Service . . . . .                      | 9         |
| 5.4 Controller . . . . .                   | 9         |
| 5.5 Image du code source . . . . .         | 10        |
| <b>6 Captures d'écran de l'application</b> | <b>11</b> |
| 6.1 Interface principale . . . . .         | 11        |
| 6.2 Fonctionnalités réalisées . . . . .    | 11        |
| <b>Conclusion</b>                          | <b>12</b> |

# Présentation du projet

L'objectif principal du projet est de créer une application web simple permettant la gestion des produits.

Chaque produit contient les informations suivantes :

- Identifiant ;
- Nom ;
- Description ;
- Prix.

# Architecture de l'application

L'application respecte une architecture MVC et une architecture en couches.

**Client** → **Controller** → **Service** → **DAO** → **Hibernate** → **MySQL**

## 2.1 Conception générale



FIGURE 2.1 – Conception générale de l'application

# Structure du projet

La structure du projet est organisée comme suit :

- **controller** : contient le contrôleur Spring MVC ;
- **service** : contient la logique métier ;
- **dao** : contient l'accès aux données avec Hibernate ;
- **model** : contient l'entité Produit ;
- **Pages** : contient les pages JSP ;
- **WEB-INF** : contient les fichiers de configuration.

# Configuration du projet

## 4.1 Fichier pom.xml

Le fichier `pom.xml` contient les dépendances nécessaires pour Spring MVC, Hibernate, MySQL et JSP/JSTL.

```
<dependency>
  <groupId>org.springframework</groupId>
  <artifactId>spring-webmvc</artifactId>
  <version>5.3.30</version>
</dependency>

<dependency>
  <groupId>org.hibernate</groupId>
  <artifactId>hibernate-core</artifactId>
  <version>5.6.15.Final</version>
</dependency>

<dependency>
  <groupId>com.mysql</groupId>
  <artifactId>mysql-connector-j</artifactId>
  <version>8.4.0</version>
</dependency>
```

## 4.2 Fichier web.xml

Le fichier `web.xml` permet de déclarer le `DispatcherServlet` de Spring MVC.

```
<servlet>
  <servlet-name>spring</servlet-name>
  <servlet-class>
    org.springframework.web.servlet.DispatcherServlet
  </servlet-class>
  <init-param>
    <param-name>contextConfigLocation</param-name>
    <param-value>/WEB-INF/application-servlet-config.xml</param-value>
  </init-param>
  <load-on-startup>1</load-on-startup>
</servlet>
```

## 4.3 Configuration Hibernate

```
<hibernate-configuration>
  <session-factory>

    <property name="hibernate.connection.driver_class">
      com.mysql.cj.jdbc.Driver
    </property>

    <property name="hibernate.connection.url">
      jdbc:mysql://localhost:3306/gestion_produits
    </property>

    <property name="hibernate.connection.username">root</
      property>
    <property name="hibernate.connection.password"></property>

    <property name="hibernate.dialect">
      org.hibernate.dialect.MySQL8Dialect
    </property>

    <property name="hibernate.hbm2ddl.auto">update</property>
    <property name="hibernate.show_sql">true</property>

    <mapping class="model.Produit"/>

  </session-factory>
</hibernate-configuration>
```

# Description des composants

## 5.1 Model

La classe `Produit` représente l'entité principale de l'application.

```
@Entity
public class Produit {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long idProduit;

    private String nom;
    private String description;
    private Double prix;
}
```

## 5.2 DAO

La classe `ProduitDao` permet de communiquer avec la base de données à travers Hibernate.

```
public class ProduitDao {

    private static SessionFactory factory;

    static {
        factory = new Configuration().configure().
            buildSessionFactory();
    }

    public void saveProduit(Produit p) {
        Session session = factory.openSession();
        session.beginTransaction();
        session.save(p);
        session.getTransaction().commit();
        session.close();
    }
}
```

## 5.3 Service

La classe `ProduitService` contient la logique métier de l'application.

```
public class ProduitService {  
  
    private ProduitDao dao = new ProduitDao();  
  
    public void addProduit(Produit p) {  
        dao.saveProduit(p);  
    }  
  
    public List<Produit> getAllProduits() {  
        return dao.getAllProduits();  
    }  
}
```

## 5.4 Controller

Le contrôleur reçoit les requêtes HTTP et retourne la vue JSP.

```
@Controller  
public class HomeController {  
  
    private ProduitService service = new ProduitService();  
  
    @RequestMapping("/index")  
    public String index(Model model) {  
        model.addAttribute("listeProduit", service.getAllProduits());  
        return "produits";  
    }  
  
    @RequestMapping("/addProduct")  
    public String addProduct(Produit p) {  
        service.addProduit(p);  
        return "redirect:/index";  
    }  
}
```

## 5.5 Image du code source

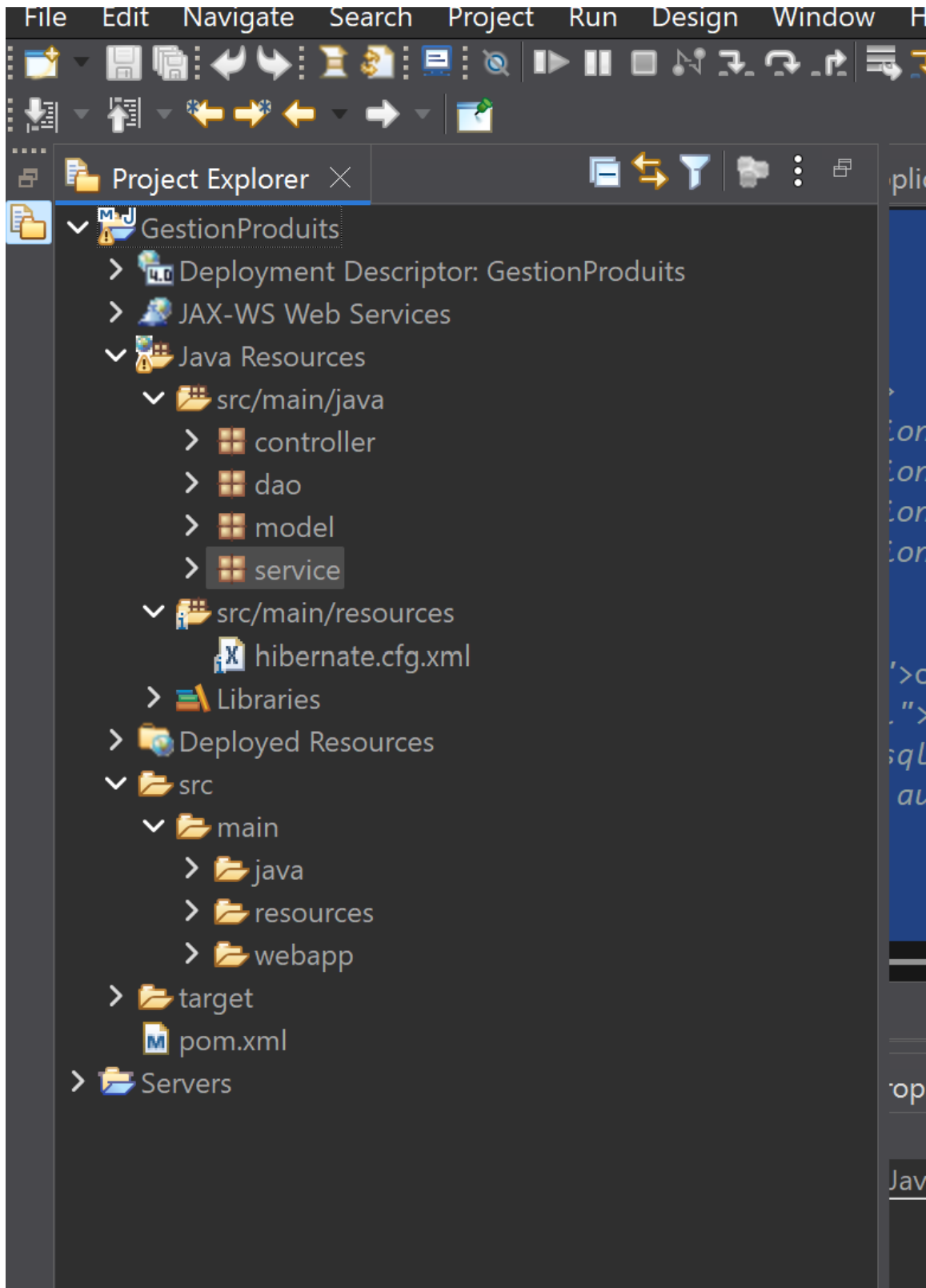


FIGURE 5.1 – Extrait du code source de l'application

# Captures d'écran de l'application

## 6.1 Interface principale

**Gestion des Produits**

Nom:

Description:

Prix:

| ID | Nom  | Description | Prix   | Actions                                     |
|----|------|-------------|--------|---------------------------------------------|
| 2  | PC 1 | Sony vaio   | 8000.0 | <a href="#">Delete</a> <a href="#">Edit</a> |
| 3  | PC2  | dell        | 1000.0 | <a href="#">Delete</a> <a href="#">Edit</a> |

FIGURE 6.1 – Interface de gestion des produits

## 6.2 Fonctionnalités réalisées

L'application permet :

- L'ajout d'un produit ;
- L'affichage de la liste des produits ;
- La modification d'un produit ;
- La suppression d'un produit.

# Conclusion

Ce TP nous a permis de mettre en pratique le développement d'une application web avec Spring MVC et Hibernate. Nous avons réalisé une application CRUD complète permettant de gérer des produits avec une base de données MySQL.